

NoteWise AI

AI-Powered Document Understanding & Interactive Learning Assistant

Hackathon Project Report

1. Problem Statement

Students and self-learners routinely deal with study material that arrives in inconsistent, hard-to-search formats: scanned textbook pages, photographed handwritten class notes, lecture PDFs, and printed handouts. Unlike typed digital text, this material cannot be searched, summarized, or revised efficiently without first being read from start to finish.

This creates three everyday problems for a learner preparing for exams or interviews:

- **Time lost in manual reading:** Condensing a 10-page scanned chapter or a handwritten notebook into revision-ready notes is slow and repetitive, especially before deadlines.
- **No active recall tools:** Most study material is static text. Learners rarely convert it into self-test questions because writing MCQs manually is tedious, so revision stays passive (re-reading) instead of active (testing).
- **Content is locked inside the file:** If a learner has a doubt about something in their notes, they cannot ask a question and get an answer grounded in that specific document — they have to search the whole file manually or look the concept up elsewhere.

The underlying problem NoteWise AI addresses is: how can a learner turn any physical or scanned study material into an accurate, structured, and interactive digital study resource — automatically, and without manual retyping?

2. Existing Solution

A few categories of tools currently attempt to address parts of this problem:

- **Plain OCR tools (e.g., Google Lens, basic scanner apps):** These convert an image of text into editable text, but stop there. They perform no summarization, no question generation, and no understanding of the content — the output is just raw, often messy, text.
- **Generic AI summarizer chatbots:** General-purpose AI chat tools can summarize text if it is pasted in manually. They are not built as a document workflow — a learner must first extract the text themselves (often by retyping it), then copy it into a chat window, then repeat the process separately for notes, quizzes, and questions.
- **Flashcard / quiz-making apps:** Several apps let users manually type flashcards or MCQs. Some offer limited AI-assisted generation, but almost all require digitally typed input; they are not designed to start from a photographed or scanned page.

- **Note-taking apps (Notion, Evernote, OneNote, etc.):** These are excellent for storage and organization but are not built to understand content — they do not summarize, generate quizzes, or answer questions about what is written inside a note.

3. Limitations of the Existing Solution

Individually, each existing tool solves only a narrow slice of the workflow, forcing the learner to stitch together multiple disconnected apps. Specific limitations include:

- **Fragmented workflow:** Extracting text, summarizing it, generating quizzes, and asking questions each require a different tool, with manual copy-pasting between every step.
- **No support for handwritten or scanned input in the AI stage:** Summarizer and chat tools work only once text already exists in digital form; they cannot start from a photo of a notebook page.
- **Passive learning only:** Even after summarization, most tools leave the learner to write their own test questions, so active recall — proven to be more effective than re-reading — is rarely used.
- **No document-grounded question answering:** General chatbots answer from their own general knowledge, not strictly from the learner's own uploaded material, so answers can drift away from what the source document actually says.
- **Repetitive manual effort:** Every new note or chapter means repeating the same extract-summarize-quiz cycle by hand, which does not scale to a full semester's worth of material.

In short, no single existing solution takes a learner from a raw photo of a page to a structured, testable, and queryable study resource in one continuous pipeline.

4. Proposed Solution

NoteWise AI is proposed as a single, end-to-end study companion that converts any uploaded study material — image, scanned document, or PDF — into an interactive learning resource, without requiring the learner to manually retype or reorganize anything.

The complete workflow implemented in NoteWise AI is:

Study Material → OCR / Text Extraction → AI/NLP Processing → Summary / Notes → MCQ Generation → Chat with Notes → Interactive Learning

What differentiates NoteWise AI from a basic AI summarizer:

- **Unified pipeline, not a single feature:** A basic summarizer only compresses text that is already typed. NoteWise AI starts one step earlier, from an image or scanned page, and carries the same extracted content through summarization, quiz generation, and Q&A — all inside one workspace, with no manual re-entry between stages.
- **Grounded, not generic, answers:** The "Chat with Notes" feature answers questions using the content of the specific uploaded document rather than open-ended general knowledge, keeping responses relevant to what the learner actually studied.

- **Built for active recall:** MCQ generation is a first-class output of the pipeline (not an afterthought), so every uploaded note automatically becomes a self-test, encouraging active revision instead of passive re-reading.
- **Multi-format, multi-language intake:** The system accepts JPG, PNG, WEBP, BMP, TIFF, PDF, and DOCX inputs, and supports selecting an input and output language, so material in one language can be studied and revised in another.
- **Transparent extraction:** The extracted OCR text is shown to the user alongside a confidence indicator before it is used for downstream AI processing, so the learner can trust — or manually verify — what the AI is working from.

Together, these design choices turn NoteWise AI from a single utility into a complete study loop: capture material once, and receive notes, key concepts, a quiz, and a chat assistant, all derived from that same source.

5. Key Features

- **Multi-format document intake:** Upload a photo, scan, PDF, or DOCX file, or capture a page directly using a device camera.
- **OCR-based text extraction with confidence scoring:** Extracts text from image-based or scanned material and displays a confidence score so the reliability of the extraction is visible, not hidden.
- **AI/NLP-generated smart summaries:** Condenses long or messy source material into a clear, readable summary of the core ideas.
- **Key concept tagging:** Automatically identifies and lists the important terms and concepts found in the material, giving the learner a quick-glance index of what the content covers.
- **Structured revision notes:** Breaks the summarized content into discrete, labeled key points that are easier to revise than a single block of text.
- **Automatic quiz generation:** Converts the understood content into a set of question-and-answer pairs the learner can use to self-test, with answers revealed on demand.
- **Chat with Notes:** Lets the learner ask questions about their uploaded material and receive answers grounded in that specific document.
- **Translation support:** Translates the extracted note content into a selected output language, making the tool usable across different native languages.
- **Listen / audio playback:** Converts summarized content to speech, supporting auditory revision alongside reading.
- **Persistent workspace:** Uploaded material and its generated study assets (summary, concepts, notes, quiz) are organized together in a workspace rather than scattered across separate tools.

6. Technologies / Stack Used

Layer	Technology	Purpose
-------	------------	---------

Frontend	HTML, CSS, JavaScript	Upload interface, Study Material views, and dynamic rendering of AI-generated results
Backend	Python, Flask	Orchestrates the pipeline: receives uploads, calls OCR and AI/NLP layers, returns results
Text Extraction	OCR	Converts image-based/scanned material into machine-readable text
Content Understanding	AI / NLP, Generative AI	Summarization, key concept extraction, note structuring, MCQ generation, Chat with Notes
Data Persistence	Database	Stores uploaded notes and their generated summaries, concepts, notes, and quiz data
Prototyping	Google Colab	Independent development and testing of OCR/AI logic before backend integration
Version Control	GitHub	Source code management and team collaboration

This stack was chosen to keep the project lightweight and fast to iterate on within a hackathon timeframe, while still demonstrating a genuine end-to-end AI pipeline: Flask provides a minimal but capable backend to orchestrate OCR and AI calls; plain HTML/CSS/JS keeps the frontend simple and dependency-free; and Google Colab was used for prototyping and testing the OCR and NLP logic before wiring it into the Flask app.

7. Implementation Details

7.1 Frontend

The frontend is built using HTML, CSS, and JavaScript and is organized around three views that mirror the pipeline: a Workspace view for uploading or capturing a note, a Study Material view that renders the AI-generated outputs (Smart Summary, Key Concepts, Revision Notes, Quiz Generator, and Translate), and a How It Works view that explains the pipeline to a new user. JavaScript handles the file upload flow, sends the file to the backend, and dynamically renders the returned OCR text, summary, key concepts, and quiz cards without a full page reload.

7.2 Backend

The backend is implemented in Python using the Flask web framework, which exposes endpoints that receive the uploaded file, coordinate the OCR and AI/NLP processing steps, and return structured results (extracted text, summary, key concepts, notes, quiz questions/answers, and translated content) to the frontend as the pipeline output. Flask was chosen for its minimal setup overhead, which suited the constrained hackathon build time while still supporting a clear separation between the upload, processing, and response stages.

7.3 OCR / Text Extraction Layer

When a file is uploaded, it is first passed through an OCR engine to convert image-based or scanned content into raw machine-readable text. Because handwritten and low-quality scanned input is inherently harder to read accurately than clean typed text, the extracted output is displayed back to the user as "Verified OCR" together with a confidence indicator, so the reliability of the extraction stage is visible rather than assumed. This raw OCR output becomes the input to the AI/NLP stage.

7.4 AI / NLP Processing Layer

The extracted text is passed to an AI/NLP layer (built using Generative AI models) that performs the core understanding tasks: generating a concise smart summary, identifying key concepts/terms, restructuring the content into discrete revision key points, generating question–answer pairs for the quiz feature, and answering learner questions for the Chat with Notes feature by reasoning over the extracted document text. Keeping this as a distinct layer from OCR means the AI is always working on the already-extracted text, so the same processing logic can be reused regardless of whether the original source was a photo, scan, PDF, or DOCX file.

7.5 Database

A database layer stores uploaded notes together with their generated study assets — OCR text, summary, key concepts, revision notes, and quiz data — so a learner's workspace persists across sessions instead of being regenerated or lost after every upload. This also underpins the Study Material view, which needs to retrieve and display a specific note's full set of generated content.

7.6 Integration Flow

The five layers connect into a single request-driven pipeline: the frontend uploads a file to the Flask backend; the backend routes it through the OCR layer to obtain raw text; that text is sent to the AI/NLP layer, which returns the summary, key concepts, notes, and quiz data in one coordinated step; the results are persisted in the database against that note; and the frontend then fetches and renders the stored results in the Study Material view, where the learner can also trigger on-demand actions like Translate, Listen, or Chat with Notes against the same stored text.

7.7 Development & Prototyping

Google Colab was used during development to prototype and validate the OCR extraction and AI/NLP summarization/question-generation logic independently of the web application, before integrating the finalized logic into the Flask backend. GitHub was used for version control and collaboration across the team throughout the build.

8. Results / Output

The following screenshots are taken directly from the working NoteWise AI application and illustrate the end-to-end pipeline in action, from uploading a handwritten note to receiving a summary, key concepts, revision notes, an auto-generated quiz, and a translated version of the extracted content.

8.1 Workspace — Uploading a Note

The Workspace view is the entry point of the pipeline. A learner can either upload an existing file (photo, scan, PDF, or DOCX) or capture a page live using a camera. The interface also lets the user select an input and output language before processing begins.

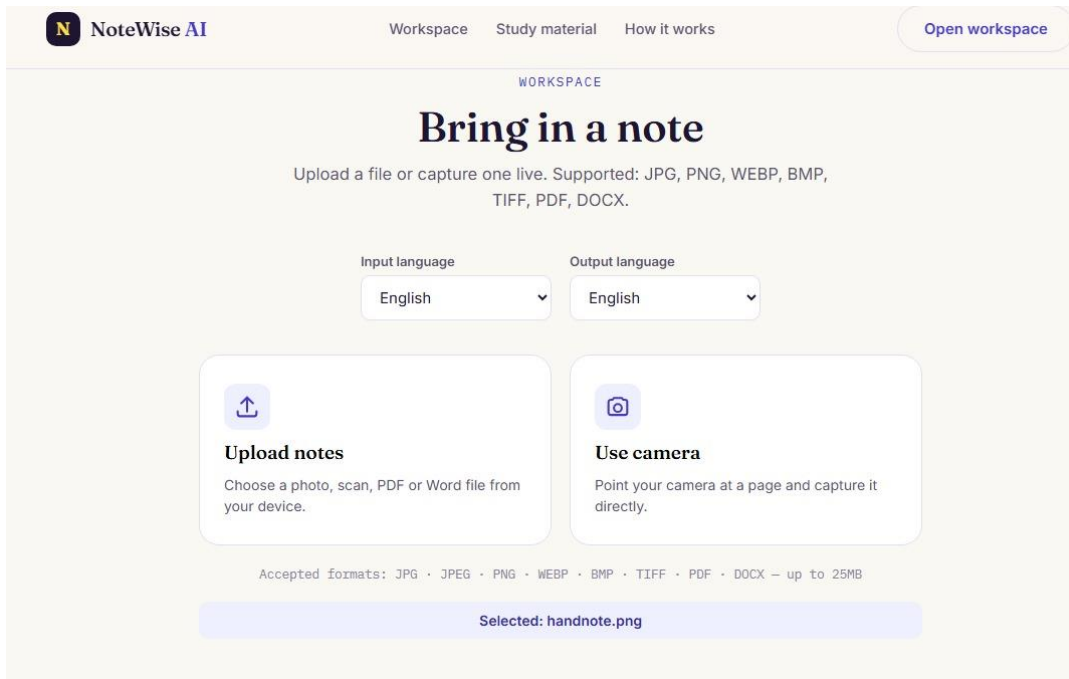


Fig. 1 — Workspace: uploading a handwritten note (handnote.png) for processing.

8.2 OCR Extraction Output

Once uploaded, the note is passed through the OCR engine. The extracted text is displayed as "Verified OCR" alongside a confidence score, and the same extracted text is stored as the note's content for downstream AI processing. In this sample, the source was a handwritten page on artificial intelligence, and the OCR layer visibly reflects the real-world difficulty of reading handwriting — the confidence score is shown transparently rather than hidden from the user.

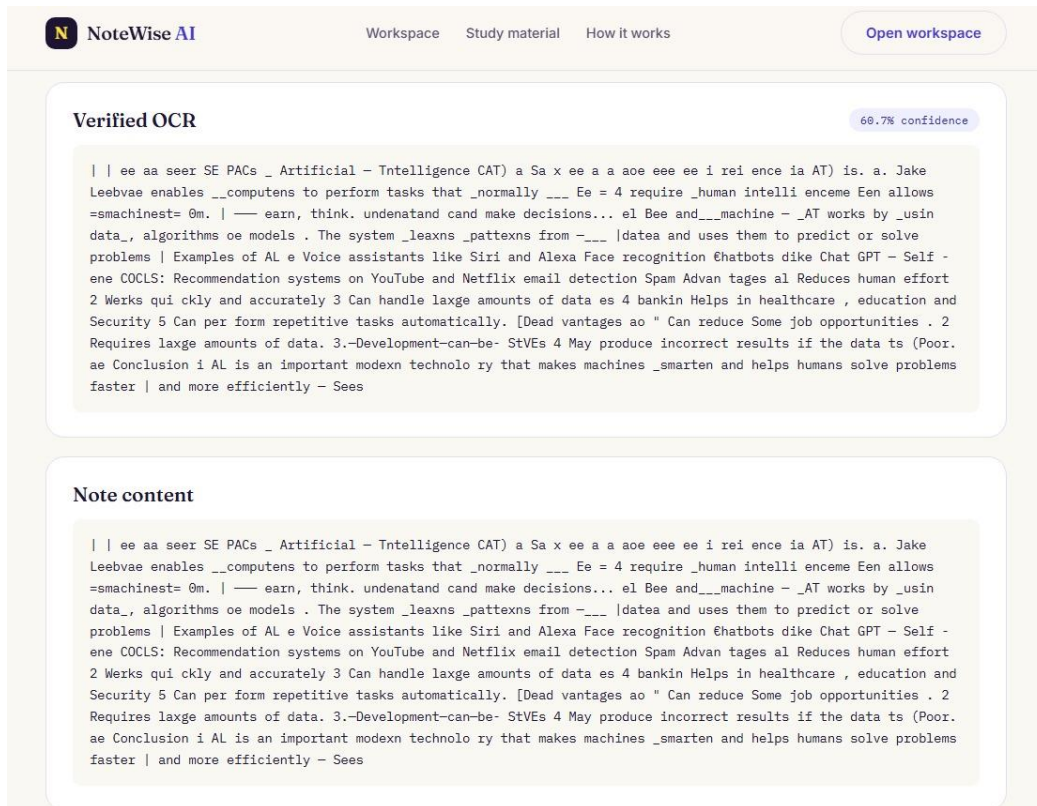


Fig. 2 — Verified OCR output and stored note content, with confidence score displayed.

8.3 Smart Summary, Key Concepts & Revision Notes

From the extracted text, the AI/NLP layer generates a Smart Summary of the material, a set of tagged Key Concepts for quick reference, and structured Revision Notes broken into individual key points. An audio playback control ("Listen" / "Generate speech") is also available for auditory revision of the summary.

Smart summary

Jake Leebvae enables __computens to perform tasks that _normally __ Ee = 4 require _human intelli enceme Een allows
=smachinest= 0m. The system _leaxns _pattexns from __ |datea and uses them to predict or solve problems | Examples of
AL e Voice assistants like Siri and Alexa Face recognition €hatbots dike Chat GPT — Self - ene COCLS: Recommendation
systems on YouTube and Netflix email detection Spam Advan tages al Reduces human effort 2 Werks qui ckly and accurately 3
Can handle laxge amounts of data es 4 bankin Helps in healthcare , education and Security 5 Can per form repetitive tasks
automatically. 2 Requires laxge amounts of data. ae Conclusion i AL is an important modexn technolo ry that makes machines
_smarten and helps humans solve problems faster | and more efficiently — Sees

Listen

Generate speech

0:08 / 1:18

Key concepts

Data

Problems

Amounts

Tasks

Human

Solve

Laxge

Helps

Recommendation

Automatically

Revision notes

KEY POINT 1

|| ee aa seer SE PACs _ Artificial — Tntelligence CAT) a Sa x ee a a aoe eee ee i rei ence ia AT) is.

KEY POINT 2

Jake Leebvae enables __computens to perform tasks that _normally __ Ee = 4 require _human intelli enceme Een allows

Fig. 3 — Smart Summary, Key Concepts, and Revision Notes generated from the uploaded note.

8.4 Auto-Generated Quiz

The same understood content is converted into a self-test Quiz Generator, presenting one question per key concept with an answer that can be revealed on demand — turning passive notes into an active-recall study tool without any manual question writing.

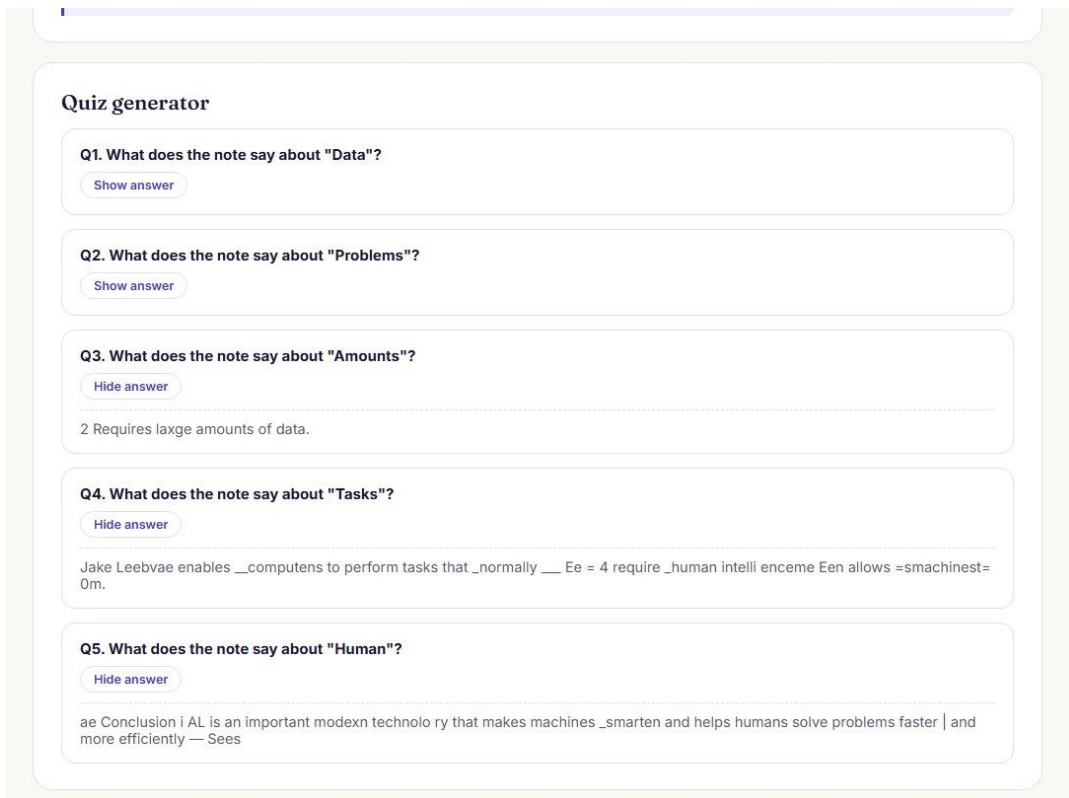


Fig. 4 — Auto-generated quiz questions with reveal-on-demand answers.

8.5 Translation & Pipeline Overview

The extracted note content can be translated into the selected output language directly within the Study Material view. The application also presents a four-step "Capture → Extract → Understand → Study" overview, summarizing the full pipeline for the end user.

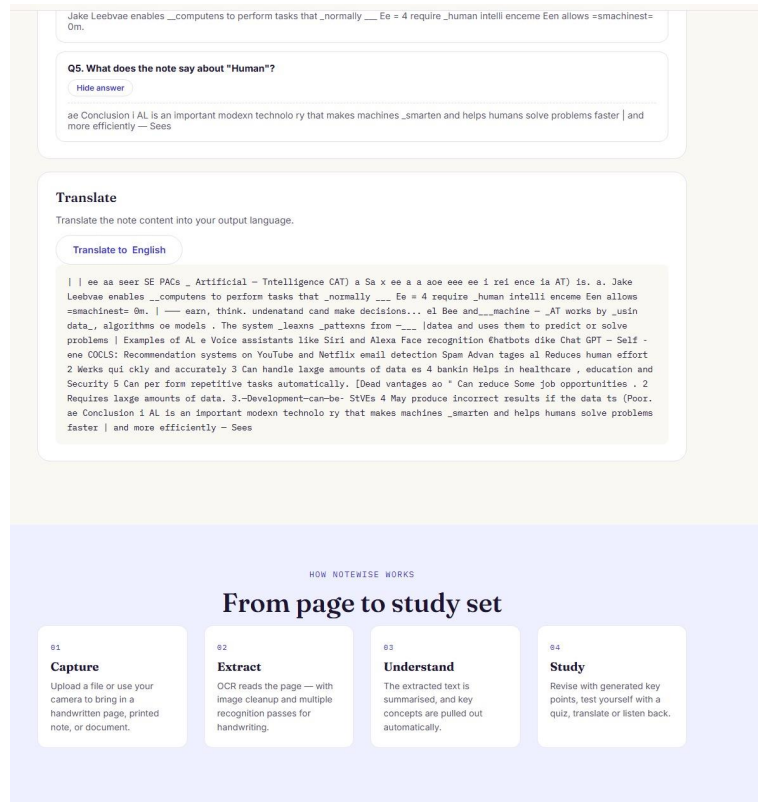


Fig. 5 — Translate feature and the four-stage NoteWise AI pipeline overview.

Across these outputs, the working prototype demonstrates that a single uploaded image can be carried through OCR, summarization, concept extraction, quiz generation, and translation within one connected workspace — validating the core proposed workflow. The current OCR confidence level on difficult handwritten input also honestly highlights an area for continued improvement, discussed in the next section.

9. Future Enhancements

- **Improved handwriting OCR accuracy:** Fine-tuning or combining multiple OCR engines/models specifically for handwritten input to raise extraction confidence on difficult, low-quality, or cursive notes.
- **Multi-document and cross-note chat:** Extending "Chat with Notes" to answer questions across an entire subject or multiple uploaded documents at once, rather than one note at a time.
- **Adaptive quiz difficulty:** Generating quizzes with varying difficulty levels and question types (MCQ, short answer, true/false) based on how a learner has performed previously.
- **Spaced-repetition scheduling:** Using stored quiz performance to automatically resurface weaker key concepts for review at optimal intervals.
- **Collaborative workspaces:** Allowing a group of learners (e.g., a class) to share and jointly build a note workspace for a course.
- **Offline/mobile-first capture:** A dedicated mobile app with offline OCR capture for classroom note-taking without requiring an internet connection at capture time.

- **Richer export options:** Exporting generated notes and quizzes to PDF, Anki-compatible flashcards, or common note-taking apps.

10. Conclusion

NoteWise AI demonstrates that the fragmented, manual process learners currently rely on — extracting text, summarizing it, writing self-test questions, and separately looking up doubts — can be replaced with a single connected pipeline: Study Material → OCR/Text Extraction → AI/NLP Processing → Summary/Notes → MCQ Generation → Chat with Notes → Interactive Learning.

By combining OCR, AI/NLP-based summarization, automatic quiz generation, and document-grounded chat within one workspace, the project moves beyond what a basic AI summarizer offers: it turns any photographed or scanned page into a structured, testable, and queryable study resource with minimal manual effort from the learner.

The working prototype — built with Python, Flask, HTML/CSS/JavaScript, OCR, and Generative AI, backed by a database and version-controlled on GitHub — validates the core pipeline end to end on real uploaded material, as shown in the Results section. With continued improvement to OCR accuracy on handwritten input and the planned enhancements outlined above, NoteWise AI has clear potential to grow from a hackathon prototype into a genuinely useful daily study companion for students dealing with scattered, non-digital study material.